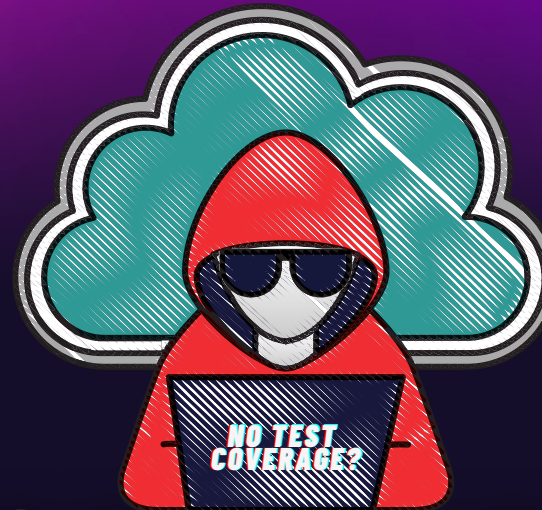




BOOTCAMP QA

HACKEANDO LA CALIDAD 2025

Clase 7

- Postman y usos del mismo.
- Workspaces: Trabajo colaborativo
- Colecciones: Estructura de armado de APIs
- Requests: Envío de solicitudes a endpoints.
- Creación de casos de prueba en Postman



DESARROLLANDO CALIDAD, STEP BY STEP

¿QUÉ ES POSTMAN?

Postman es una herramienta de desarrollo API que permite enviar solicitudes HTTP a servidores y recibir respuestas para probar, documentar y colaborar en el desarrollo, consulta y testing de APIs.



¿PARA QUE SIRVE POSTMAN EN EL TESTING?

Ejecutar casos de prueba de APIs REST.

Validar condiciones para cada caso de prueba.

Organizar colecciones para la prueba de multiples casos de prueba al mismo tiempo

Automatizar la carga de datos en ambientes.

Analizar estructura de los request creados que luego utilizara el Front End team.

Analizar estructura de los request creados que luego utilizara el Front End team.

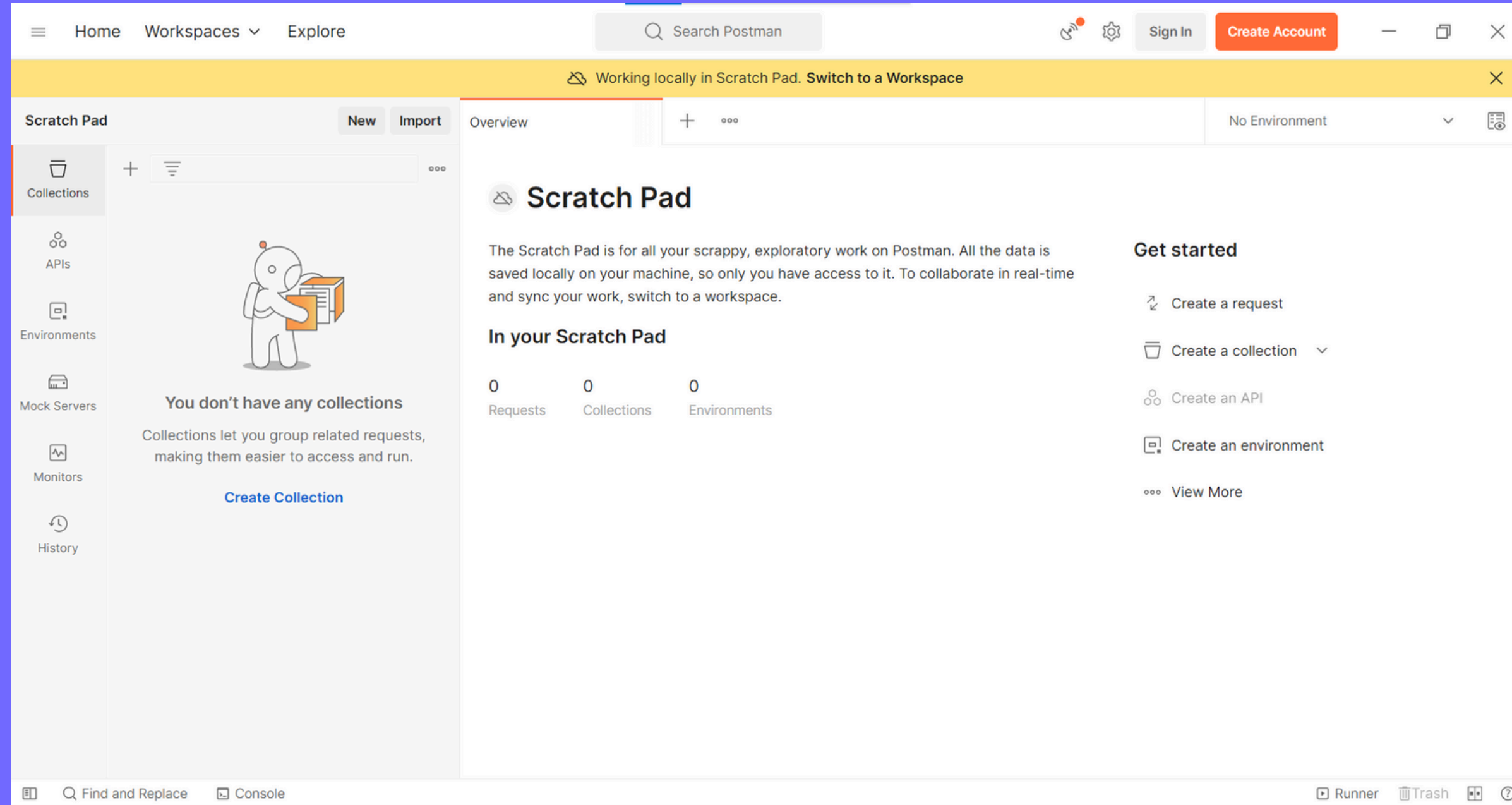


¿PARA QUE LE SIRVE POSTMAN A UN TESTER QA?

Postman permite realizar pruebas y validar APIs de manera eficiente. Les permite enviar solicitudes HTTP, recibir respuestas y realizar pruebas automatizadas para verificar el comportamiento y rendimiento de las APIs.



POSTMAN – DESKTOP



POSTMAN – CREAR UN WORKSPACE

←

→

Home

Workspaces ▾

API Network ▾

Explore

🔍 Search Postman

⚙️

🔔



Upgrade ▾

—

📄

✕

Create workspace

Name

Summary

Add a brief summary about this workspace.

Visibility

Determines who can access this workspace.

☐ Personal

Only you can access

☐ Private

Only invited team members can access

☒ Team

All team members can access

☐ Partner NEW

Only invited partners and team members can access

☐ Public

Everyone can view

📘

A team of your own will be created since you don't have one now.

Create Workspace and Team

Cancel



FUNCIONAMIENTO POSTMAN

Script de Precondiciones

Variables

**Envio del request al
endpoint**

API

**Recepcion del
response**

Ejecucion de los tests

**Escritura posterior de
variables**

FUNCIONAMIENTO POSTMAN

1. Workspace (Espacio de trabajo)

Es donde agrupas tus colecciones, entornos, y pruebas. Puedes tener espacios de trabajo personales o compartidos con un equipo.

2. Sidebar (Barra lateral)

Incluye accesos rápidos a:

- Collections (colecciones): Agrupaciones de solicitudes.
- APIs: Documentación de APIs.
- Environments: Variables para diferentes entornos (dev, staging, prod).
- History: Registro de solicitudes realizadas.
-

3. Request Builder (Constructor de solicitudes)

Zona central para crear una solicitud:

Método HTTP: **GET, POST, PUT, DELETE**, etc.

URL: Dirección del **endpoint**.

Params: Parámetros de consulta (query params).

Headers: Encabezados personalizados.

Body: Datos enviados (form, raw, JSON, etc.).

Auth: Métodos de autenticación (Bearer Token, Basic Auth, etc.).

4. Response Viewer (Visor de respuestas)

Muestra la respuesta del servidor:

- Body: JSON, HTML, texto, etc.
- Status: Código HTTP (200 OK, 404 Not Found...).
- Time: Tiempo de respuesta.
- Size: Tamaño de la respuesta.
- Headers: Encabezados recibidos.

5. Console (Consola de Postman)

Útil para depurar. Muestra detalles de solicitudes y respuestas, errores y console.log() si usas scripts.

6. Tests y Pre-request Scripts

En la pestaña Tests puedes escribir scripts en JavaScript para validar respuestas.

En Pre-request Script puedes ejecutar código antes de enviar la solicitud.

URL API – ENDPOINT

The screenshot displays the Postman interface for a REST client. The top navigation bar includes links for Home, Workspaces, API Network, and Explore, along with a search bar and an 'Invite' button. The main workspace is titled 'AcademiaQA - Clase Postman' and shows a collection named 'Basic API test' with a folder 'User' containing a 'POST Create User' endpoint. The endpoint URL is 'http://security.postman-breakable.com/user'. The request body is a JSON object:

```
{  "username": "mariano1234",  "password": "mariano1234"}
```

. The response is a 200 OK status with a response time of 380 ms and a body size of 275 B. The response body is displayed in the 'Body' tab, showing a JSON object:

```
{  "response": {    "user_id": "160d33e1-8d63-4552-970e-ace048e993f3",    "username": "mariano1234"  },  "status": "OK"}
```



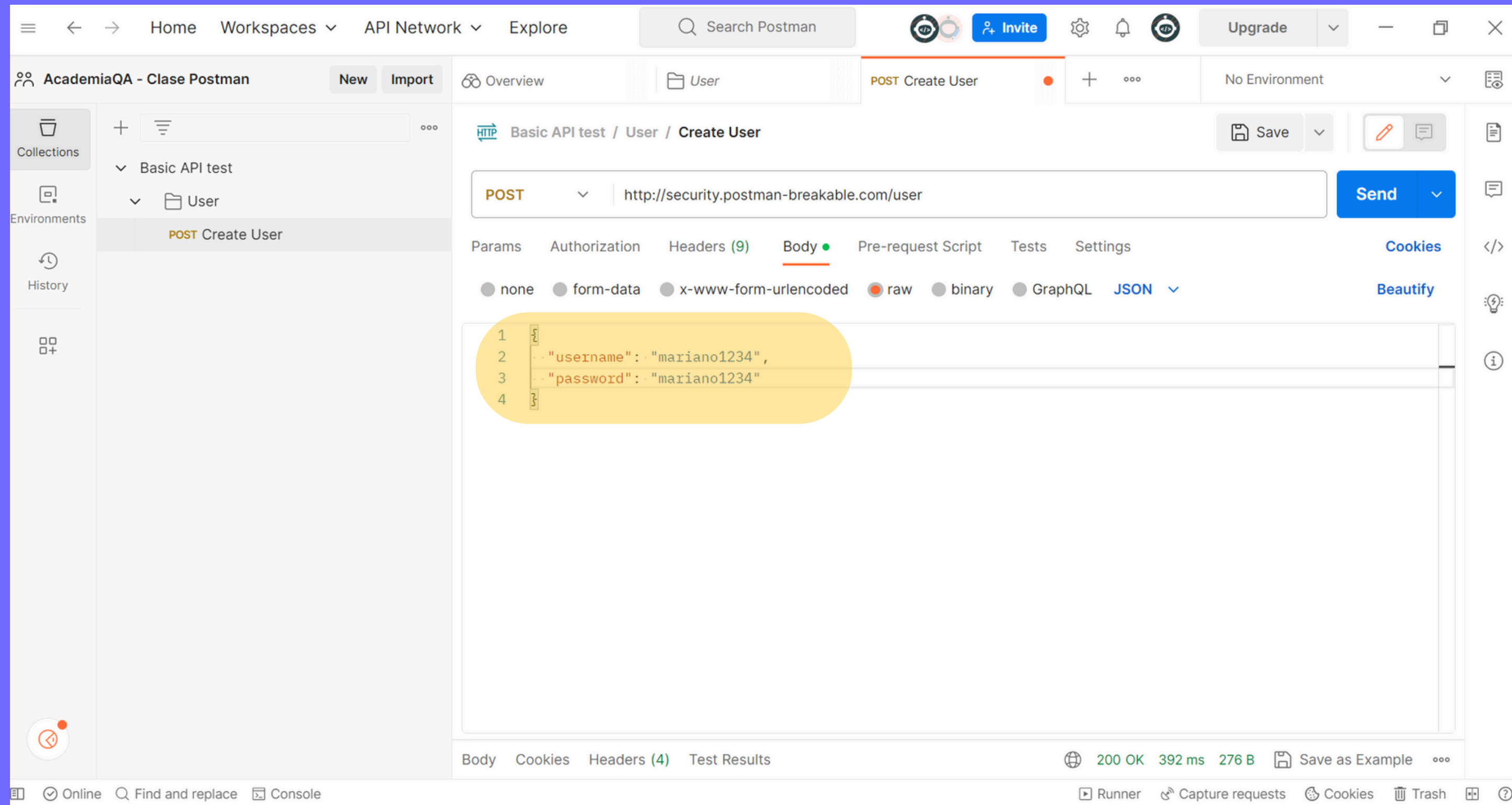
CAMPOS DE UN REQUEST – HEADER

The screenshot displays the Postman interface for a POST request named 'Create User' within a collection 'Basic API test'. The 'Headers' tab is selected, showing a list of 10 default headers. A yellow rounded rectangle highlights these headers. The status bar at the bottom indicates the request is 'Online'.

Key	Value	Description
☒ Cache-Control	no-cache	
☒ Postman-Token	<calculated when request is sent>	
☒ Content-Type	application/json	
☒ Content-Length	<calculated when request is sent>	
☒ Host	<calculated when request is sent>	
☒ User-Agent	PostmanRuntime/7.32.2	
☒ Accept	/*/*	
☒ Accept-Encoding	gzip, deflate, br	
☒ Connection	keep-alive	
☒		



CAMPOS DE UN REQUEST – BODY



RESPONSE 200 – OK

The screenshot displays the Postman interface for a REST client. The workspace is named "AcademiaQA - Clase Postman". The active collection is "Basic API test", and the selected environment is "User". The request is a POST to "http://security.postman-breakable.com/user". The request body is a JSON object with "username" and "password" both set to "mariano1234". The response is a 200 OK status with a response time of 380 ms and a body size of 275 B. The response body is displayed in the "Body" tab, showing a JSON object with "response" and "status" fields.

Request Details:

- Method: POST
- URL: http://security.postman-breakable.com/user
- Body (JSON):

```
{  1  {  2    "username": "mariano1234",  3    "password": "mariano1234"  4  }
```

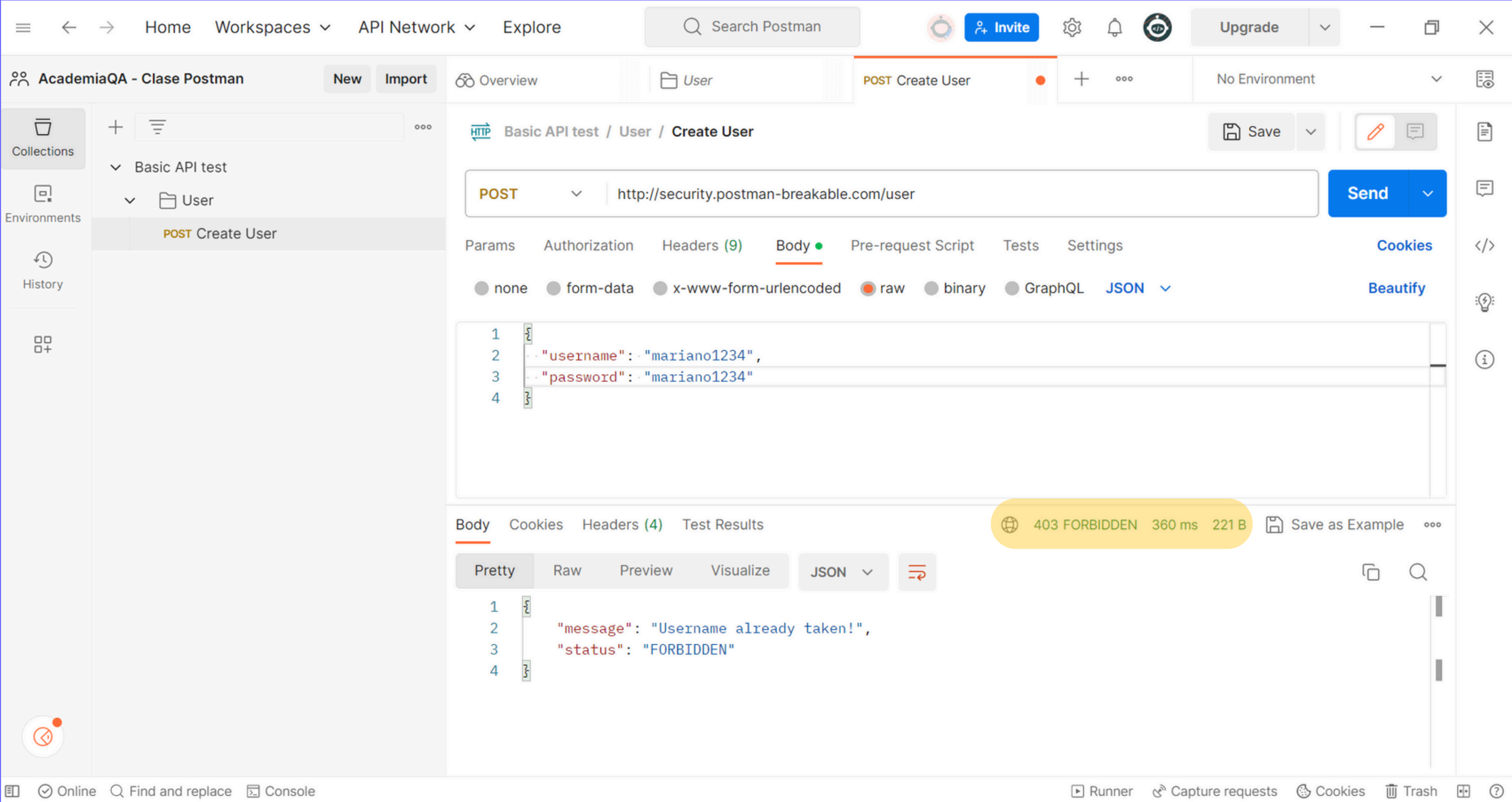
Response Details:

- Status: 200 OK
- Time: 380 ms
- Size: 275 B
- Body (JSON):

```
1  {  2    "response": {  3      "user_id": "160d33e1-8d63-4552-970e-ace048e993f3",  4      "username": "mariano1234"  5    },  6    "status": "OK"  7  }
```



RESPONSE 403 – FORBIDDEN



CONSOLE LOG

AcademiaQA - Clase Postman

NewImport

Overview

User

POST Create User

No Environment

Collections

Basic API test

User

POST Create User

History

POST

http://security.postman-breakable.com/user

Send

Params

Authorization

Headers (10)

Body

Pre-request Script

Tests

Settings

Cookies

Content-Type

Content-Length

Host

User-Agent

application/json

<calculated when request is sent>

<calculated when request is sent>

PostmanRuntime/7.32.2

Body

Cookies

Headers (4)

Test Results

403 FORBIDDEN

654 ms

221 B

Save as Example

Online

Find and replace

Console

All Logs

Clear

403

654 ms

Show raw log

POST http://security.postman-breakable.com/user

Network

Request Headers

Request Body

Response Headers

Response Body



CONSOLE LOG – REQUEST HEADER

The screenshot shows the Postman interface with a POST request to `http://security.postman-breakable.com/user`. The request headers are displayed in the console log, and the response status is 403 FORBIDDEN.

Request Headers:

- Content-Type: "application/json"
- User-Agent: "PostmanRuntime/7.32.2"
- Accept: "*/*"
- Cache-Control: "no-cache"
- Postman-Token: "7e84956e-fa91-4176-aad7-8f70bc4a97ce"
- Host: "security.postman-breakable.com"
- Accept-Encoding: "gzip, deflate, br"
- Connection: "keep-alive"
- Content-Length: "63"

Response: 403 FORBIDDEN (654 ms, 221 B)



CONSOLE LOG – REQUEST BODY

The screenshot displays the Postman interface for a workspace named "AcademiaQA - Clase Postman". The active collection is "Basic API test", and the selected item is "User / Create User". The request is a POST to "http://security.postman-breakable.com/user". The "Headers" tab is selected, showing three headers: "Content-Type" (application/json), "Content-Length" (<calculated when request is sent>), and "Host" (<calculated when request is sent>). The console log at the bottom shows the request details, with the "Request Body" highlighted in a yellow box. The request body is a JSON object: { "username": "mariano1234", "password": "mariano1234" }. The response status is "403 FORBIDDEN" with a response time of "654 ms" and a size of "221 B".

AcademiaQA - Clase Postman

New Import Overview

Basic API test / User / Create User

POST http://security.postman-breakable.com/user

Save Send

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies

Content-Type application/json

Content-Length <calculated when request is sent>

Host <calculated when request is sent>

Body Cookies Headers (4) Test Results

403 FORBIDDEN 654 ms 221 B Save as Example

Online Find and replace Console

POST http://security.postman-breakable.com/user

Network

Request Headers

Request Body

```
{
  "username": "mariano1234",
  "password": "mariano1234"
}
```

Response Headers

Response Body

Runner Capture requests Cookies Trash



CONSOLE LOG – RESPONSE BODY

The screenshot shows the Postman interface with a workspace named "AcademiaQA - Clase Postman". The active collection is "Basic API test" and the environment is "User". The selected request is a POST request to "http://security.postman-breakable.com/user". The request headers are:

Header	Value
Content-Type	application/json
Content-Length	<calculated when request is sent>
Host	<calculated when request is sent>

The response status is 403 FORBIDDEN, with a response time of 654 ms and a body size of 221 B. The console log shows the response body:

```
{
  "message": "Username already taken!",
  "status": "FORBIDDEN"
}
```



CONSOLE LOG – RESPONSE BODY

Basic API test

Share

Fork | 0

0

Run

Save

...

Overview

Authorization ●

Pre-request Script

Tests

Variables ●

Runs

These variables are specific to this collection and its requests. Learn more about [collection variables](#) ↗

Filter variables

	Variable	Initial value	Current value	...
☒	url	http://security.postman-b...	http://security.postman-breakable.com	
	Add new variable			





CONFIGURACIÓN DE VARIABLES

The screenshot displays the Postman interface for a workspace named "AcademiaQA - Clase Postman". The left sidebar shows a collection "Basic API test" with a sub-collection "User" containing several endpoints. The main panel is configured for a POST request to "Basic API test / User / Create User". The request body is a JSON object with "username" and "password" fields, both set to "mmariano12345". The response panel shows a successful status of 200 OK with a JSON body containing "response" and "status" fields.

Request Configuration:

- Method: POST
- URL: {{url}}/user
- Body Type: JSON
- Body Content:

```
1 {
2   "username": "mmariano12345",
3   "password": "mmariano12345"
4 }
```

Response:

- Status: 200 OK
- Time: 383 ms
- Size: 277 B
- Body Type: JSON
- Body Content:

```
1 {
2   "response": {
3     "user_id": "5988d918-ae3b-480d-a321-d787e5d0182c",
4     "username": "mmariano12345"
5   },
6   "status": "OK"
7 }
```





TEST CASE 1 – GET

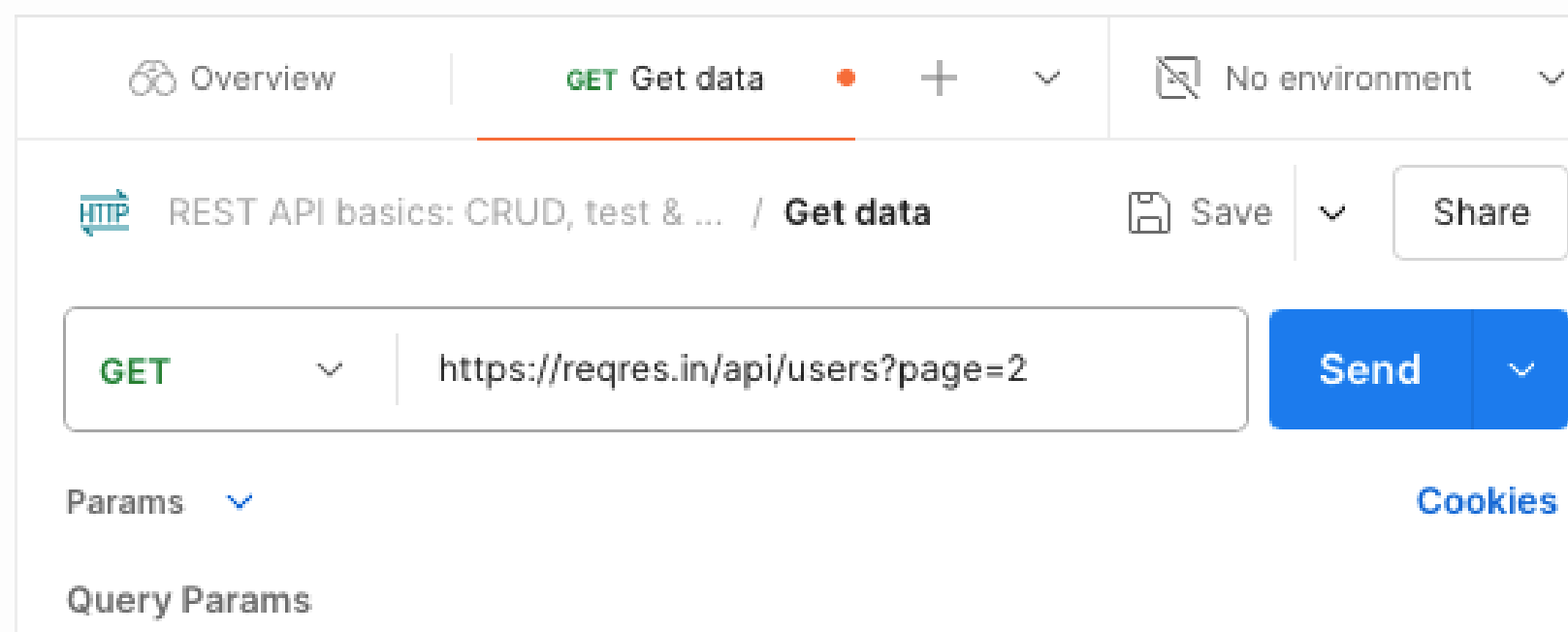
Titulo: Verificar funcionamiento del endpoint

Pasos:

1. Apuntar a <https://reqres.in/api/users?page=2> utilizando un metodo GET.
2. Obtener API Key <https://reqres.in/>
3. Ingresar API Key en Headers
4. Presionar SEND.

Resultado esperado:

- Respuesta 200 OK.





TEST CASE 2 – POST

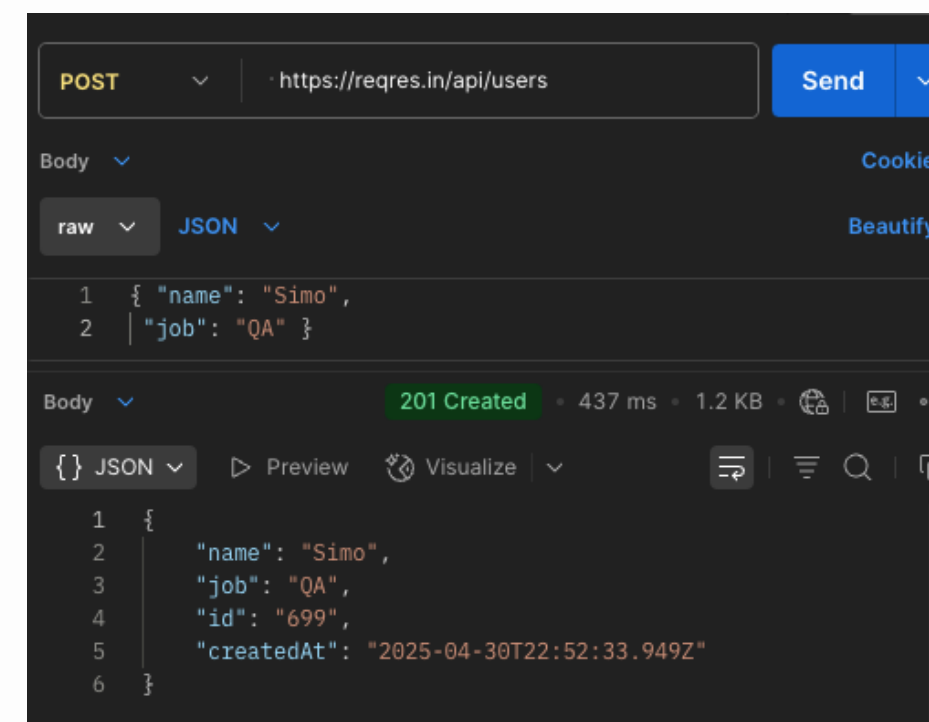
Título: Creación nuevo usuario en el sistema por medio de metodo http post.

Pasos:

1. Apuntar a <https://reqres.in/api/users> utilizando un metodo POST.
2. Obtener API key <https://reqres.in/signup>
3. En Body seleccionamos "RAW" y luego "JSON" pegar credenciales:
{ "name": "Simo",
 "job": "QA" }
4. Presionar SEND.

Resultado esperado:

Código Respuesta 201 Created.





TEST CASE 3 PUT

Título: Actualizar un ID de usuario

Pasos:

1. Introducir la URL <https://regres.in/api/users/>
2. Seleccionar el método HTTP PUT
3. Introducir el Body seleccionando el formato "raw" y json.
4. {"name": "morpheus",
"job": "zion resident"}

Resultado esperado:

Código Respuesta 200 Ok





SWAGGER

(DOCUMENTADO COMO SI FUERA HECHO POR UN BACKEND)

<https://petstore.swagger.io>

 **Swagger**
Supported by SMARTBEAR

Explore

Swagger Petstore

1.0.6 OAS 2.0

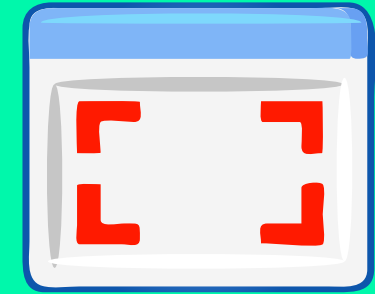
[Base URL: petstore.swagger.io/v2]
<https://petstore.swagger.io/v2/swagger.json>

This is a sample server Petstore server. You can find out more about Swagger at <http://swagger.io> or on [#swagger](irc://freenode.net). For this sample, you can use the api key **special-key** to test the authorization filters.

[Terms of service](#)
[Contact the developer](#)
[Apache 2.0](#)
[Find out more about Swagger](#)



PORTAFOLIO



En tu portafolio agrega **una captura de pantalla de la ejecución de alguno de los TestCases**





Contact Us



<http://www.AcademiaQA.com>



[@academia_qa](https://twitter.com/academia_qa)



info@academiaqa.com



[@academia_qa](https://www.instagram.com/academia_qa)



<https://t.me/academiaqa/2242>



<https://www.youtube.com/c/AcademiaQAOK>



<https://www.linkedin.com/company/academiaqa/>

academia**qa**.com



academ,

